

자료구조 요약 정리: 큐, 덱, 배열 리스트, 연결 리스트

1. 선형 큐 (Linear Queue)

기본 개념

- FIFO(First In First Out) 구조
- 삽입은 rear에서, 삭제는 front에서
- 초기 상태: `front = rear = -1`

주요 연산

- **isEmpty()**: `front == rear`
- **isFull()**: `rear == MAX_SIZE - 1`
- **enqueue(item)**: rear 증가 후 삽입
- **dequeue()**: front 증가 후 제거

예시

```
int queue[MAX];
int front = -1, rear = -1;

// enqueue
if (rear < MAX - 1) queue[++rear] = 10;

// dequeue
if (front != rear) int val = queue[++front];
```

2. 원형 큐 (Circular Queue)

기본 개념

- 배열을 원형으로 사용해 공간 활용도 높임
- 한 칸은 항상 비워서 `isFull`과 `isEmpty` 구분
- 초기 상태: `front = rear = 0`

주요 연산

- **isEmpty()**: `front == rear`
- **isFull()**: `(rear + 1) % MAX == front`
- **enqueue(item)**: `rear = (rear + 1) % MAX, queue[rear] = item`
- **dequeue()**: `front = (front + 1) % MAX, return queue[front]`

예시

```
int queue[MAX];
int front = 0, rear = 0;

// enqueue
if ((rear + 1) % MAX != front)
    queue[rear = (rear + 1) % MAX] = 10;

// dequeue
if (front != rear)
    int val = queue[front = (front + 1) % MAX];
```

3. 덱 (Deque: Double Ended Queue)

기본 개념

- 양쪽에서 삽입과 삭제 가능
- 연산 종류:
 - `addFront, deleteFront`
 - `addRear, deleteRear`

주요 연산

- `isFull(): (rear + 1) % MAX == front`
- `isEmpty(): front == rear`

예시

```
// 구조체 기반 구현 시
addFront(item): front = (front - 1 + MAX) % MAX
addRear(item): rear = (rear + 1) % MAX
deleteFront(): front = (front + 1) % MAX
deleteRear(): rear = (rear - 1 + MAX) % MAX
```

4. 배열 기반 리스트

특징

- 고정 크기 배열 사용
- 삽입/삭제 시 이동 필요 → 비효율적

삽입 예시

```
for (int i = size; i > pos; i--)
    list[i] = list[i - 1];
```

```
list[pos] = item;
size++;
```

삭제 예시

```
for (int i = pos; i < size - 1; i++)
    list[i] = list[i + 1];
size--;
```

5. 연결 리스트 (Linked List)

기본 개념

- 노드(Node): 데이터 + 포인터
- 포인터를 통해 노드끼리 연결됨
- 동적 메모리 사용 (`malloc`, `free`)

장점

- 삽입/삭제 효율적 (포인터만 조정)
- 크기 제한 없음

단점

- 구현 복잡, 포인터 오류 발생 가능성 있음

구조 예시 (C 코드)

```
typedef struct Node {
    int data;
    struct Node* next;
} Node;

Node* head = NULL;

// 삽입 (맨 앞)
Node* newNode = (Node*)malloc(sizeof(Node));
newNode->data = 10;
newNode->next = head;
head = newNode;

// 삭제 (맨 앞)
if (head != NULL) {
    Node* temp = head;
    head = head->next;
    free(temp);
}
```

6. 연결 리스트 용어 정리

- **Node**: 데이터 필드 + 링크 필드로 구성
- **Head**: 리스트의 시작 포인터
- **NULL**: 마지막 노드의 링크 값

시각적 예시

```
[10|*] → [20|*] → [30|NULL]
^head
```

참고

- 큐 응용 예: 작업 스케줄링 (Job Scheduling), 시뮬레이션 시스템, 고객 대기열 등
- 연결 리스트 응용 예: 메모리 관리, 그래프 표현, 스택/큐 구현 등

마무리 팁

- 큐와 리스트의 핵심은 **포인터 조작**과 **인덱스 계산**에 있음
 - 원형 큐는 모듈러 연산 %이 중요함
 - 연결 리스트는 **malloc/free** 사용 연습 필요함
 - 에러 처리를 꼭 넣어야 안정적인 코드 완성됨
-